



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240218

姓 名： 杨陆续

实验（1）

【实验目的】

1. 掌握 Git 与 GitHub 的基本使用流程，包括账号注册、仓库创建、项目初始化和版本控制操作。
2. 熟悉一个标准项目的初始化流程，包括配置文件（如.gitignore,.env）和项目说明文档（README.md）的创建与管理。

【实验内容】

1. 注册 GitHub 账号，并完成基本配置。
2. 在 GitHub 上创建一个公开（public）的代码仓库（Repository），仓库名与 GitHub 账号名保持一致，此仓库将作为个人首页。
3. 在个人首页仓库中，创建并编写 README.md 文件，展示个人介绍或项目信息。
4. 本地初始化一个项目，关联远程 GitHub 仓库，并完成首次代码推送（Push）。
5. 理解并实践项目的标准初始化步骤，包括创建.gitignore、.env等配置文件。

1.2 实验过程：

1. 项目初始化与 Git 操作：注册 GitHub 账号后，在平台上创建了名为[YourGithubUsername]的公开仓库。在本地，通过 git init 命令初始化新项目，创建了基本的项目结构。通过 git remote add

origin 命令将本地仓库与远程仓库关联。

2. 配置文件创建：在项目根目录下创建了 README.md 文件，用于描述项目目标和后续更新日志。创建了 .gitignore 文件，用于排除不需要版本控制的文件（如虚拟环境目录、IDE 配置文件、.env 等）。创建了 .env 文件模板 (.env.example)，用于说明项目所需的环境变量。
3. 代码提交与推送：将项目文件添加到 Git 暂存区（git add .），提交初始版本（git commit -m "Initial commit"），并推送到 GitHub 远程仓库（git push -u origin main）。

1.2 实验结果：



jiangcheng1997 / README.md

Hi there 🤖 你好呀!

我是一名计算机/软件工程/电子信息（根据你的实际专业修改）专业的大二学生，正在努力把代码写得更好一点 🚀

- 🔭 我正在努力完成：专业课作业 + 一个小项目（比如：待办清单App / 个人博客 / 校园助手）
- 🌱 我正在学习：数据结构与算法 / Python / Java / 前端基础（选你正在学的）
- 🤝 我想一起合作：课设项目 / 开源新手任务 / 组队参加比赛（如“互联网+”、服创大赛）
- 💡 我需要帮助：如何高效刷LeetCode / 如何写出更规范的代码
- 💬 可以问我：大二怎么选方向、Git/GitHub基础、期末复习方法
- 👉 称呼：随便叫~
- 🔥 有趣的事：我经常看着自己的代码笑出来（不是因为写得好，是因为太奇怪了😂）

★ 欢迎来看看我的小程序，一起交流学习~

Popular repositories

Repository	Language	Visibility
tmp	Python	Public
jiangcheng1997		Public

【实验总结】

本次实验完成了从环境准备到项目上线的完整流程。通过实践，我掌握了使用 Git 进行版本控制的核心命令，理解了 README.md、.gitignore、.env 等文件在项目管理中的重要作用。实验为后续的代码开发和协作奠定了基础，强调了及时更新文档和提交代码的良好习惯。

实验（2）

【实验目的】

1. 学习并部署本地大语言模型（LLM）应用环境，包括 AnythingLLM 和 LM Studio 的安装与配置。
2. 掌握在本地使用私有化 LLM 与文档进行交互的基本方法。
3. 理解本地向量数据库和上下文压缩的基本概念。

【实验内容】

1. 下载并安装 AnythingLLM（一个本地部署的 LLM 应用平台）。
2. 配置 LM Studio 作为本地模型服务，并将其连接到 AnythingLLM。
3. 在 AnythingLLM 中创建新的工作区（Workspace），并上传个人文档（如实验报告、学习笔记等）。
4. 通过配置好的环境，与上传的文档进行问答互动。
5. 学习上下文压缩和本地查询的基本技巧。

2.1 实验过程：

1. 环境部署：从官方渠道下载 AnythingLLM 安装包并完成安装。同时，下载并运行 LM Studio，在 LM Studio 中加载一个合适的开源大语言模型（如 Qwen、Llama 等系列模型）并启动本地 API 服务。
2. 应用配置：在 AnythingLLM 的设置中，将模型提供商配置为“LM Studio”，并填入 LM Studio 提供的本地 API 地址（通常为 `http://localhost:1234/v1`）。

3. 文档交互：在 AnythingLLM 中创建一个新的工作区。通过“添加文档”功能，将本地的实验报告文档（如.docx 或.txt 文件）上传至工作区。上传后，系统会自动对文档进行切片和向量化处理，存入内置的向量数据库。
4. 功能测试：在工作区的聊天界面中，针对上传的文档内容进行提问。例如，询问“实验一的目的是什么？”，系统将基于向量数据库检索相关文档片段，并调用 LM Studio 中的模型生成回答。

2.2 实验结果：



【实验总结】

本次实验让我亲身体验了在本本地私有化部署大语言模型应用的完整流程。通过集成 AnythingLLM 和 LM Studio，实现了在不依赖云端 API 的情况下，安全、私密地与个人文档进行智能交互。我理解了向量数据库如何将文档转化为可查询的知识，以及上下文管理在长文档对话中的重要性。这为后续开发更复杂的 AI 应用提供了实践基础。

实验（3）

【实验目的】

1. 开发具备自动聊天历史总结与压缩能力的 AI 对话代理（Agent）。
2. 实现基于规则的关键信息提取与本地日志记录功能。
3. 开发通过自然语言指令查询聊天历史日志的功能。

【实验内容】

1. 基于实验二的环境，开发新的对话代理功能：
2. 上下文总结与压缩：当对话轮次超过 5 轮或上下文总长度超过 3000 字符（token）时，自动触发 LLM 对聊天历史进行总结压缩（压缩前 70% 的内容，保留后 30% 的原始内容），以维持对话上下文的高效性。
3. 关键信息提取与记录：每进行 5 轮对话，自动触发 LLM 提取对话中的关键信息，并按照 5W 原则（Who, What, When, Where, Why）进行结构化记录，增量保存到本地文件 D:\chat-log\log.txt 中。
4. 历史日志查询：开发一个功能调用（function call），当用户输入以“/search”开头，或表达“查找聊天历史”意图时，能够读取 log.txt 文件，结合用户查询，由 LLM 给出整合后的回答。

3.1 实验过程：

1. 项目结构搭建：在项目目录下新建 practice03 文件夹，并将实验二的相关基础代码迁移至此作为新代码 1 的基础。
2. 功能一开发（新代码 1）：

1. 在对话处理循环中，增加对聊天历史轮次和上下文长度的监控逻辑。
 2. 当触发条件（>5 轮或>3k 长度）满足时，构造特定的提示词（Prompt），调用 LLM API，要求其对“前 70%”的聊天内容进行总结，生成一个简短的摘要。
 3. 用生成的摘要替换掉原有上下文中对应的部分，形成新的、压缩后的上下文，用于后续对话。
3. 功能二与三开发（新代码 2）：
1. 关键信息提取：设置计数器，每 5 轮对话后，构造提示词要求 LLM 根据 5W 规则提取多条关键信息。在代码中实现检查并创建目录 `D:\chat-log\`` 和文件 `log.txt`` 的逻辑，并将提取到的信息以追加（append）模式写入该文件。
 2. 历史查询功能：开发一个新的函数（如 `search_chat_log`）。在对话入口处，检测用户输入：a) 是否以 `/search` 开头；b) 通过一个简单的意图分类器或直接让 LLM 判断是否包含查询历史的意图。若触发，则读取 `log.txt` 文件内容，将其与用户问题一起作为新的上下文提交给 LLM，请求其进行信息整合与回答。
4. 测试与文档更新：对两个新功能进行完整测试，确保上下文压缩、日志记录和查询功能均能正确运行。测试无误后，更新项目根目录的 `README.md` 文件，详细说明 `practice03` 中新增的功能和使用方法。

3.2 实验结果:

根据聊天记录内容，按照 5W 规则提取的关键信息如下：

```
**1. 关键信息一（用户身份与互动）**
*   **Who:** 陶正雨
*   **What:** 向 AI 助手打招呼并进行自我介绍。
*   **When:** 未明确提及（聊天开始时）。
*   **Where:** 未明确提及（线上对话场景）。
*   **Why:** 为了开启对话并建立初步联系。

**2. 关键信息二（宠物情况）**
*   **Who:** 陶正雨
*   **What:** 养有一条小狗。
*   **When:** 持续进行。
*   **Where:** 未明确提及。
*   **Why:** 为了陪伴或作为生活乐趣分享（推断）。

**3. 关键信息三（个人特征与身份）**
*   **Who:** 陶正雨
*   **What:** 描述自己拥有长头发，并说明性别为女生。
*   **When:** 聊天过程中。
*   **Where:** 未明确提及。
*   **Why:** 为了补充个人信息画像（自我介绍补充）。

**4. 关键信息四（饮食喜好）**
*   **Who:** 陶正雨
*   **What:** 表示喜欢吃西瓜。
*   **When:** 聊天结尾时。
*   **Where:** 未明确提及。
*   **Why:** 为了表达个人口味偏好。
[关键信息提取完成]
[创建目录：D:\chat-log]
[关键信息已写入：D:\chat-log\log.txt]
```

```
成功加载 .env 文件：d:\tmp\.env
API URL: http://localhost:1234/v1
模型名称：Qwen3.5-4b
```

```
你：/search 我叫什么名字
执行工具：search_chat_history
解析后的参数：{}
助手：
```

```
根据您提供的聊天历史记录，结合第 1 点关键信息（用户身份与互动），您的名字是**陶正雨**。
```

```
你：█
```

【实验总结】

本次实验是提示工程与 AI 应用开发的综合实践。通过设计和实现特定的提示词与逻辑，我让基础的 LLM 对话能力具备了“记忆管理”（上下文压缩）和“长期记忆”（关键信息日志）的高级功能。我深入理解了如何通过程序逻辑与 LLM 提示词配合，来解决实际应用中的上下文长度限制和历史信息检索难题。实验巩固了 Git 操作流程，强化了开发-测试-文档更新-代码提交的完整工作习惯。